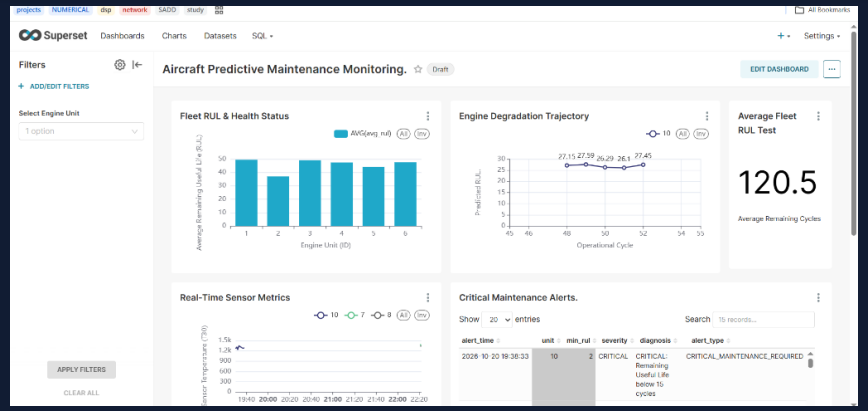
BIG DATA ENGINEERING & PREDICTIVE ANALYTICS

Aircraft Engine Health Monitoring & Predictive Maintenance

An End-to-End Big Data Pipeline Using NASA N-CMAPSS, Apache Spark, Kafka, HDFS, Airflow, MLlib, ClickHouse & Apache Superset

LIVE DASHBOARD PREVIEW (APACHE SUPERSET)



WHAT THIS PROJECT DOES (SYSTEM SUMMARY)

- **Goal:** Predict engine Remaining Useful Life (RUL) and detect high-temperature faults early.
- **Data Source:** NASA N-CMAPSS turbofan engine sensor data (17.46 million rows / 6.12 GB).
- **Two Processing Paths:** Historical Batch processing (HDFS + Spark) and Streaming replay (Kafka).
- **Final Output:** Machine learning predictions and real-time alerts in ClickHouse and Superset.

KEY TECHNICAL FEATURES

- **Batch + Streaming:** Combines full history analysis with fast stream monitoring.
- **No Data Leakage:** Splits training and testing data by engine ID.
- **Docker Containers:** All 11 services run in isolated Docker containers.

17.46M Rows
NASA N-CMAPSS Data
DS01 (7.64M) & DS03 (9.82M)

RMSE: 6.36
Random Forest Model
R^2 = 0.945 on Unseen Engine

10-Task DAG
Apache Airflow
Automated Pipeline Steps

ClickHouse
Database & Dashboard
600K Predictions & Alerts

Problem Statement: Why Aircraft Engine Monitoring Needs Big Data

Understanding engine failures, maintenance costs, and the challenge of processing massive sensor data

1. UNEXPECTED ENGINE FAILURES

- **Safety & Flight Delays:** Sudden engine problems during operation cause major safety risks and costly flight cancellations.
- **Different Flight Stress:** Every engine wears out at a different speed depending on altitude, speed, and weather.
- **Early Warning Signs:** Small changes in temperature and pressure appear many flights before an engine actually fails.

2. HIGH MAINTENANCE COSTS

- **Fixing Too Late (Reactive):** Waiting until a part breaks causes severe engine damage and expensive emergency repairs.
- **Fixing Too Early (Fixed Schedule):** Replacing parts on a fixed schedule wastes 20% to 40% of healthy engine life.
- **Smart Solution (Predictive):** We should perform maintenance based on the real condition of each engine.

3. THE BIG DATA CHALLENGE

- **Huge Sensor Volume:** Engines record 50 values every second—creating 17.46 million rows (6.12 GB) in our dataset.
- **Memory Crashes (OOM):** Normal tools like Excel or basic Pandas crash when loading multi-gigabyte HDF5 files into RAM.
- **Two Speed Needs:** We need heavy batch training on historical data AND fast monitoring on incoming sensor streams.

COMPARISON OF MAINTENANCE STRATEGIES

STRATEGY	WHEN DO WE FIX IT?	MAIN PROBLEM	TECHNOLOGY NEEDED
Reactive (Run-to-Failure)	Only after the engine breaks	Dangerous, causes flight delays and high repair cost	Basic fault logs only
Preventive (Fixed Schedule)	After a fixed number of flights	Wastes money by replacing parts that are still good	Simple flight counters
Predictive (Our Project)	When sensors show low Remaining Life	Needs Big Data storage, ML, and stream processing	**HDFS + Spark + Kafka + ClickHouse**

Objectives & Proposed Solution: Predicting Remaining Useful Life

How data analytics and machine learning help maintenance teams act before failures happen

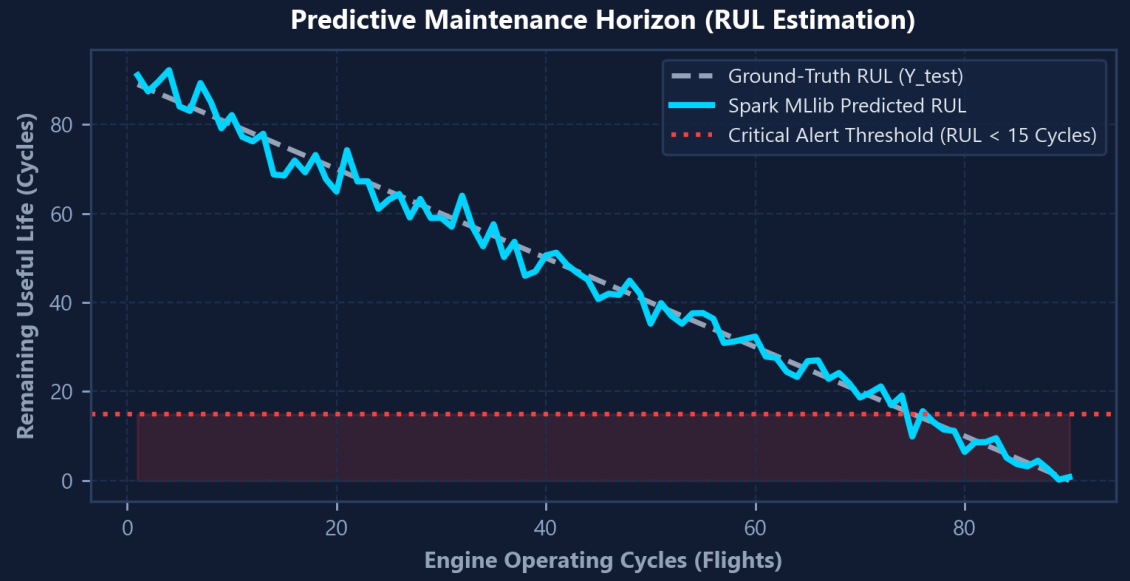
WHAT IS REMAINING USEFUL LIFE (RUL)?

- **Simple Definition:** RUL means Remaining Useful Life. It estimates how many flight cycles an engine can safely complete before it needs maintenance.
- **Important Note:** RUL is an estimated number of remaining flights based on sensor health — NOT an exact or guaranteed failure date.
- **How It Helps:** When predicted RUL drops below 15 cycles (or temperature T30 goes above 1580°R), the system sends a CRITICAL alert.

FOUR MAIN PROJECT OBJECTIVES

- **1. Safe Data Conversion:** Convert large NASA HDF5 files into fast Parquet files in 50,000-row chunks without running out of memory.
- **2. Batch & Streaming Pipelines:** Use Spark Batch for full historical analysis and Kafka + Spark Streaming for real-time alert simulation.
- **3. Accurate Machine Learning:** Train and compare 3 regression models in Spark MLlib to predict RUL without data leakage.
- **4. Automated Workflow & Dashboard:** Use Airflow to run all 10 pipeline steps automatically and show results in ClickHouse and Superset.

HOW RUL PREDICTION TRIGGERS ALERTS



WHY WE CHOSE THESE TECHNOLOGIES

- **Why Parquet?** Reads specific sensor columns fast and shrinks file size.
- **Why Kafka + Spark?** Handles high-speed sensor streams and calculates 30-second averages.
- **Why ClickHouse?** Runs dashboard queries on 600,000+ predictions in milliseconds.

End-to-End System Architecture: Batch, Streaming, ML & Dashboard

Connecting the dataset, ingestion, storage, Spark processing, machine learning, Airflow, and Superset

DIAGRAM COLORS: Batch Path (HDFS + Spark + ML) | Streaming Path (Kafka + Spark Stream) | Database & Dashboard

1. DATA SOURCE & PREP

NASA N-CMAPSS HDF5
- DS01-005 (2.68 GB)
- DS03-012 (3.44 GB)
- 50 Columns Total

▼ Convert (50K chunks)

Apache Parquet Files
- Snappy Compressed
- dev.parquet & test.parquet
- replay_sample.parquet

WHY? Prevents memory crashes and checks that real RUL labels exist.

2A. BATCH PIPELINE (STORAGE, SUMMARY & MACHINE LEARNING)

Hadoop HDFS Storage
- NameNode + DataNode
- Stores Parquet data, ML models, and predictions

→ **Spark Batch Job**
- spark_batch_summary.py
- Calculates min, max, and average RUL for each engine

→ **Spark MLlib Models**
- Splits by Engine ID
- Random Forest Winner
- Predicts RUL on 2.73M test rows

2B. STREAMING PIPELINE (HISTORICAL DATA REPLAY — NOT LIVE FLIGHT)

Python Kafka Producer
- Replays Parquet rows
- Sends 200 msgs/sec
- Adds 2% bad messages to test error handling

→ **Apache Kafka (KRaft)**
- engine-telemetry (41K)
- engine-health-results
- maintenance-alerts
- 3 Partitions per topic

→ **Spark Streaming**
- 30-sec Window stats
- Sends bad JSON to HDFS Quarantine (DLQ)
- Writes live alerts

3. DATABASE & BI

ClickHouse Database
Database: aircraft
1. engine_batch_summary
2. engine_rul_predictions
3. engine_stream_metrics
4. maintenance_alerts

▼ Fast SQL Queries

Apache Superset
- Fleet RUL Status
- Degradation Curve
- Live Sensor T30
- Critical Alerts Table

4. ORCHESTRATION & DEPLOYMENT (AIRFLOW + DOCKER)

- Apache Airflow: Runs all 10 steps automatically in order, from checking the raw HDF5 file to verifying ClickHouse data.
- Docker Compose & Swarm: All services run in Docker containers; configured for 3-node Swarm over Tailscale VPN and tested on local Docker Desktop.

Dataset & Data Preparation: NASA N-CMAPSS & HDF5-to-Parquet

Verified dataset sizes, sensor groups, data validation, and memory-safe Parquet conversion

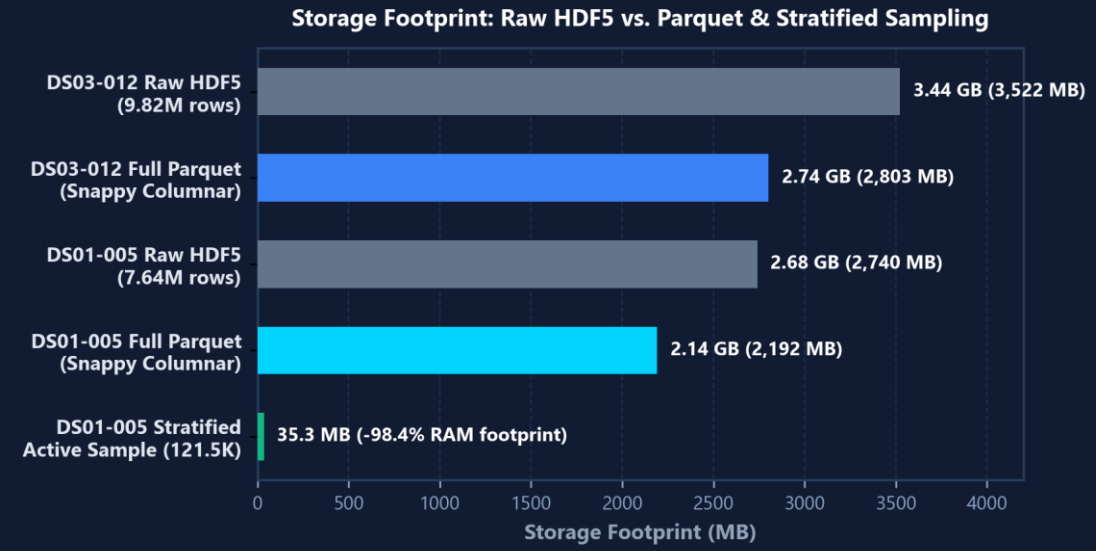
VERIFIED NASA N-CMAPSS FILES IN PROJECT

DATASET FILE	RAW SIZE	DEV SPLIT (TRAIN)	TEST SPLIT
N-CMAPSS_DS01-005.h5 (Main Pipeline Dataset)	2.68 GB	4,906,636 rows (Units 1–6, RUL 0–99)	2,735,232 rows (Units 7–10, 4 Engines)
N-CMAPSS_DS03-012.h5 (Inspected & Converted)	3.44 GB	5,571,277 rows (A_dev: 5.57M × 4 cols)	4,251,560 rows (Converted to Parquet)
Total Combined Data in Project Folder	6.12 GB	10,477,913 rows (Total Dev Rows)	6,986,792 rows (17.46M Total Rows)

WHAT IS INSIDE THE 50 COLUMNS?

- **Group A — Engine Info (4 cols):** Engine `unit` ID, flight `cycle` number, flight class (`Fc`), and health state (`hs`).
- **Group W — Flight Conditions (4 cols):** Altitude (`W_alt`), Mach speed (`W_Mach`), throttle angle (`W_TRA`), and inlet temp (`W_T2`).
- **Group X_s — Physical Sensors (14 cols):** Temperatures (`T24`, `T30`, `T48`, `T50`), pressures (`P15..Ps30`), fan/core speeds, and fuel flow.
- **Groups X_v (14), T (10) & Target Y (1):** 14 virtual sensors + 10 health parameters + real target `rul` + `dataset` & `split` = 50 columns.

FILE SIZE REDUCTION: HDF5 VS. PARQUET



HOW WE VALIDATE & CONVERT THE DATA

- **Step 1 — Data Validation:** Checks that the HDF5 file exists and has real `Y_dev` RUL values (never invents fake labels).
- **Step 2 — 50,000-Row Chunks:** Reads 50,000 rows at a time using `h5py` and writes to Parquet so RAM never overflows.
- **Step 3 — Fast Stratified Sample:** Keeps full Parquet backups and creates a ~60K-row sample (17.8 MB) for fast daily pipeline runs.

Source: NASA N-CMAPSS Dataset & Verified Project Pipeline Logs

Data Ingestion: Batch Pipeline vs. Kafka Streaming Path

Comparing historical batch loading with our Kafka streaming simulation

⚠ IMPORTANT NOTE: Our streaming path replays historical NASA Parquet rows with current timestamps using `producer.py` to test real-time processing — it is NOT live data from a flying airplane.

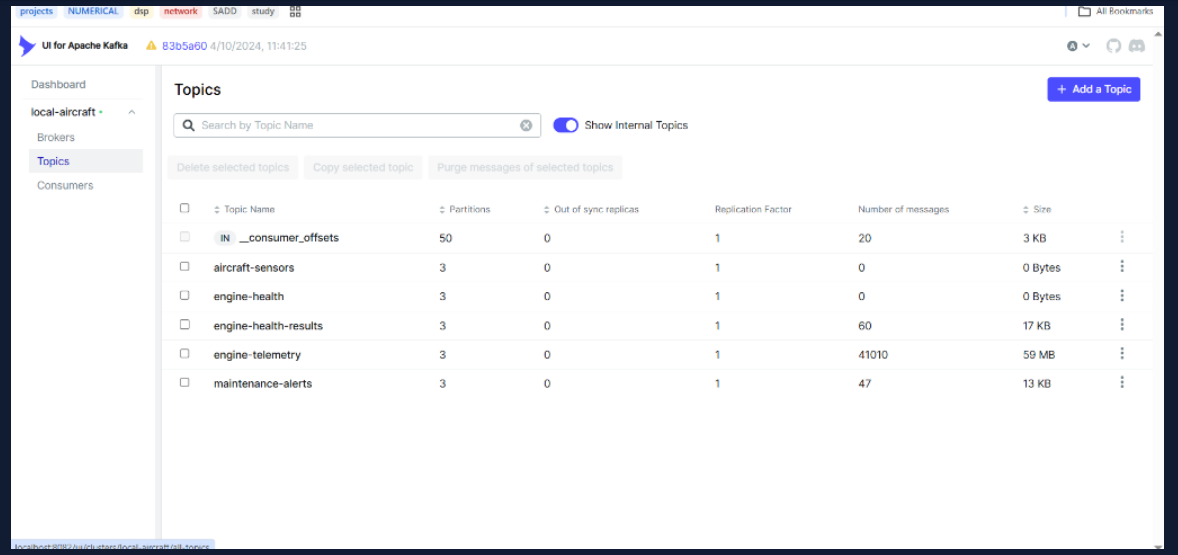
1. BATCH INGESTION (HDFS UPLOAD)

- **How It Works:** Converts HDF5 to Parquet files and uploads `dev.parquet` and `test.parquet` into Hadoop HDFS.
- **WHY We Need Batch:** Training machine learning models requires reading millions of historical rows across all engine flights at once.
- **Smart Check:** If the Parquet files are already in HDFS, the pipeline skips re-uploading to save time.

2. STREAMING INGESTION (APACHE KAFKA)

- **How It Works (`producer.py`):** Reads `replay_sample.parquet` (27,353 rows), adds a current timestamp, and sends JSON messages to Kafka at up to 200 messages/sec.
- **WHY We Need Kafka:** Kafka acts as a fast, safe buffer between the data sender and Spark Streaming, split across 3 partitions.
- **Testing Bad Data (DLQ):** Can inject 2% corrupted messages (`--inject-anomaly-rate 0.02`) to prove that bad data does not crash the stream.

VERIFIED SCREENSHOT: KAFKA UI TOPICS (PORT 8082)



VERIFIED KAFKA TOPICS & MESSAGE COUNTS

- **`engine-telemetry`:** 41,010 messages (59 MB) — incoming sensor readings.
- **`engine-health-results`:** 60 messages (17 KB) — 30-second window averages.
- **`maintenance-alerts`:** 47 messages (13 KB) — critical alerts (RUL < 15 or T30 > 1580).

Big Data Processing & Storage: Spark, HDFS, Parquet & ClickHouse

What each technology does and WHY we selected it for our architecture

1. APACHE SPARK (v4.1.3) — PROCESSING ENGINE

- **WHAT It Does:** Runs our Batch Summary job, Structured Streaming job, and Machine Learning training/prediction jobs.
- **WHY We Used Spark:** One single fast engine handles Batch, Streaming, and ML in memory—instead of needing 3 separate tools.
- **Smart Error Handling (DLQ):** In streaming mode, Spark uses 30-second windows and sends any broken JSON messages to an HDFS Quarantine folder (`Dead-Letter Queue`).

2. HADOOP HDFS (v3.2.1) — DISTRIBUTED STORAGE

- **WHAT It Does:** Stores our processed Parquet files, trained ML models (`/aircraft/models/rul_model`), predictions, and streaming checkpoints.
- **WHY We Used HDFS:** Provides central shared storage so any Spark worker container can read the data and load the saved ML model.
- **Docker Network Fix:** Configured with `use.datanode.hostname=false` so containers connect smoothly by IP address.

3. APACHE PARQUET — COLUMNAR FILE FORMAT

- **WHAT It Does:** Stores the 50-column dataset in a compressed columnar format on disk and inside HDFS.
- **WHY Parquet Instead of CSV or HDF5?** 1) Saves column data types automatically; 2) Spark only reads the 44 columns needed for ML; 3) Snappy compression makes files much smaller and faster to scan.
- **Verified Result:** Compresses millions of engine rows while keeping fast read speeds.

4. CLICKHOUSE (v26.9) — FAST ANALYTICAL DATABASE

- **WHAT It Does:** Stores the final results in 4 tables: batch summaries, 600K RUL predictions, live stream metrics, and maintenance alerts.
- **WHY ClickHouse Instead of PostgreSQL?** HDFS is great for big files, but dashboards need fast SQL queries. We upgraded from PostgreSQL to ClickHouse because its columnar `MergeTree` engine answers queries in milliseconds.
- **Safe Writing:** Both Batch and Streaming write to ClickHouse over HTTP port 8123.

Workflow Orchestration: Apache Airflow 10-Step Pipeline

Automating task order, retries, and verification with `aircraft_engine_batch_pipeline`

THE 10 AUTOMATED STEPS IN OUR AIRFLOW DAG (@daily)

Step 1 (Python) — `1_validate_raw_hdf5`
Checks HDF5 file & real Y_dev RUL (4.91M)

→

Step 2 (Bash) — `2_convert_hdf5_to_parquet`
Converts HDF5 to Parquet in 50K chunks

→

Step 3 (Bash) — `3_upload_parquet_to_hdfs`
Uploads dev & test Parquet to HDFS

→

Step 4 (Spark) — `4_spark_batch_summary`
Calculates cycle & RUL summary per unit

→

Step 5 (Spark ML) — `5_train_and_save_ml_model`
Trains 3 models & saves Random Forest

Step 6 (Spark ML) — `6_predict_rul_test_engines`
Predicts RUL on 2,735,232 test rows

→

Step 7 (ClickHouse) — `7_load_results_to_clickhouse`
Loads summaries & 50K predictions/run

→

Step 8 (Streaming) — `8_spark_structured_streaming`
Checks/starts Spark Streaming job

→

Step 9 (Kafka) — `9_kafka_producer`
Sends 1,000 sample messages @ 200/sec

→

Step 10 (Verify) — `10_clickhouse_verification`
Checks stream count in ClickHouse (57 rows)

WHY WE USED APACHE AIRFLOW

- **Runs Steps in Strict Order:** Spark ML training will never start unless raw data validation and HDFS upload finish first (`Task 1 >> Task 2 >> ... >> Task 10`).
- **Automatic Retries:** If a container is busy, Airflow waits 3 minutes and retries automatically (for example, Step 4 recovered and succeeded on retry).
- **Non-Blocking Stream Test:** Step 8 starts Spark Streaming in the background, and Step 9 sends 1,000 Kafka messages (`--no-loop --limit 1000`) so the pipeline finishes cleanly.

VERIFIED RESULTS FROM AIRFLOW LOGS

- **16 Pipeline Runs Logged:** Recorded between Oct 20 and Oct 23, 2026, stored in PostgreSQL.
- **Full 10/10 Success Verified:** Multiple runs completed all 10 steps 100%, with Step 10 returning `57` streaming rows from ClickHouse.
- **Honest Note on Latest Run (Oct 23):** Steps 1 to 9 succeeded completely. Step 10 failed (`exit code 125`) only because its command looked for network `aircraft_bigdata_swarm` while Docker Compose used `compose_default`.

Machine Learning & RUL Prediction: Spark MLlib Benchmark

How we trained, compared, and evaluated our RUL prediction models on real NASA data

1. HOW WE TRAINED THE MODELS (`ml_train.py`)

- **Target to Predict (`rul`):** True Remaining Useful Life (0 to 99 cycles across 4.91M rows in DS01-005).
- **44 Input Features:** Uses all 44 numeric sensor and flight condition columns (`VectorAssembler` + `StandardScaler`). Excludes ID columns (`unit`, `cycle`, `rul`).
- **No Data Leakage (Split by Engine ID):** We split data by engine `unit` so the same engine is NEVER in both Train and Test:

• Train Engines `[2, 3, 4, 5]`: 363,672 rows (10% sample)

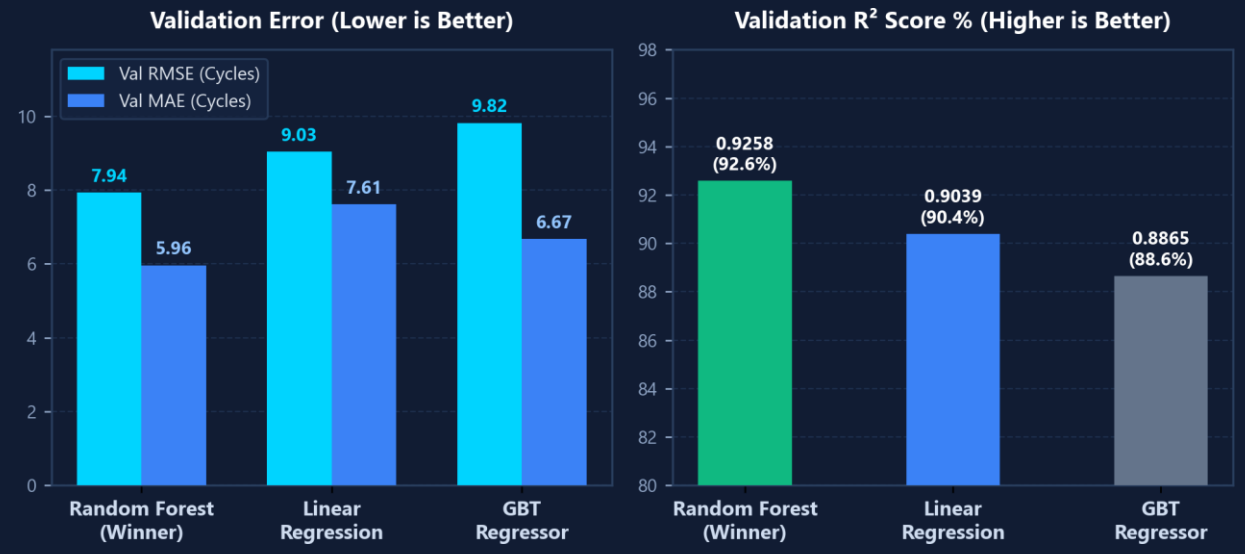
• Validation Engine `[1]`: 433,325 rows

• Unseen Test Engine `[6]`: 836,850 rows

2. VERIFIED MODEL ACCURACY TABLE

MODEL / DATASET	RMSE	MAE	R^2 SCORE
Linear Regression (Val Unit 1)	9.0328	7.6089	0.9039
GBT Regressor (Val Unit 1)	9.8152	6.6737	0.8865
Random Forest [WINNER - Val]	7.9351	5.9612	0.9258
Random Forest (Unseen Unit 6)	6.3599	5.4292	0.9451
Random Forest (Full 2.73M Test)	7.5074	5.6703	0.9059

MODEL COMPARISON CHART (FROM AIRFLOW TASK 5 LOGS)



3. PREDICTIONS ON TEST ENGINES (`ml_predict.py` — UNITS 7–10)

- **Winning Model Saved to HDFS:** `RandomForestRegressor` (25 trees, max depth 6) had the lowest error and highest R^2 score.
- **Lowest Predicted RUL per Test Engine (across 2,735,232 rows):** • Engine Unit 10 (82 cycles): Min Predicted RUL = 3.49 cycles (Most Urgent!)

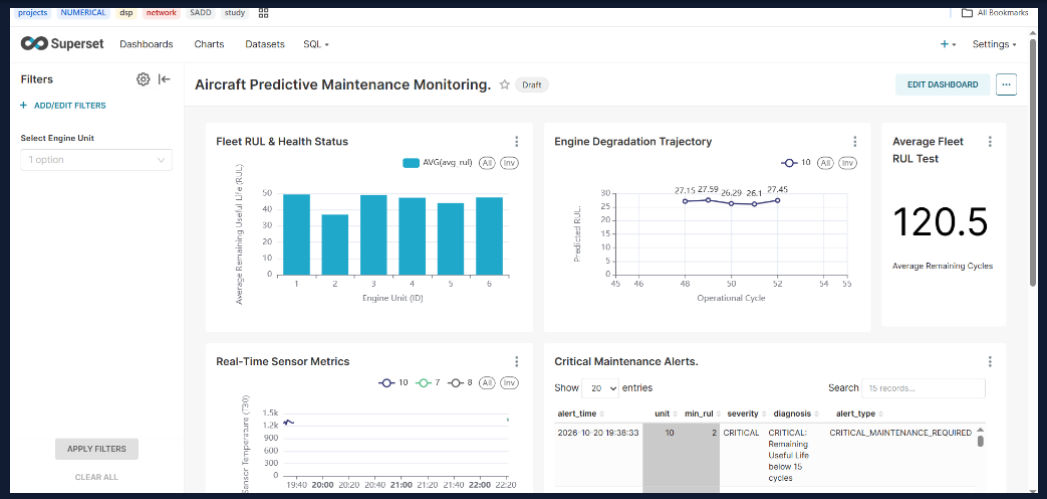
• Engine Unit 8 (89 cycles): Min Predicted RUL = 4.38 cycles

• Engine Unit 9 (80 cycles): Min Predicted RUL = 4.65 cycles | Unit 7: Min RUL = 4.90

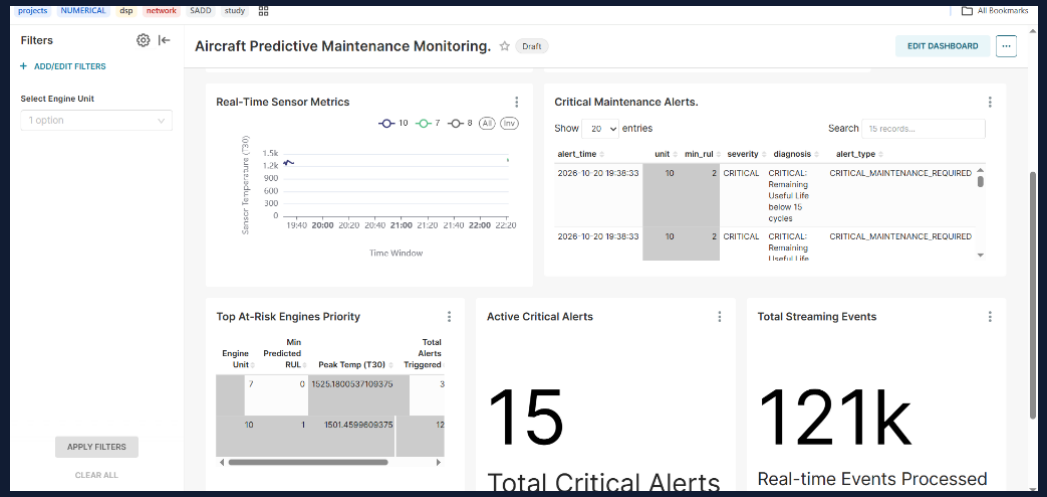
Results & Dashboard: ClickHouse Tables & Apache Superset

Verified database row counts and real screenshots from our running Superset dashboard

VERIFIED APACHE SUPERSET DASHBOARD (`localhost:8088`)



Top Dashboard Charts:
- Fleet RUL Status bar chart (Units 1–6)
- Engine 10 RUL Degradation curve
- KPI: 120.5 Average Fleet RUL



Bottom Dashboard Charts:
- Real-Time Sensor T30 temperature
- Critical Maintenance Alerts table
- 15 Active Alerts & 121k Stream Events

VERIFIED CLICKHOUSE TABLE COUNTS

TABLE NAME (`aircraft`)	ROWS	PURPOSE
`engine_batch_summary`	12	Per-engine stats
`engine_rul_predictions`	600,000	ML RUL predictions
`engine_stream_metrics`	57	30s stream windows
`maintenance_alerts`	15	Critical alerts

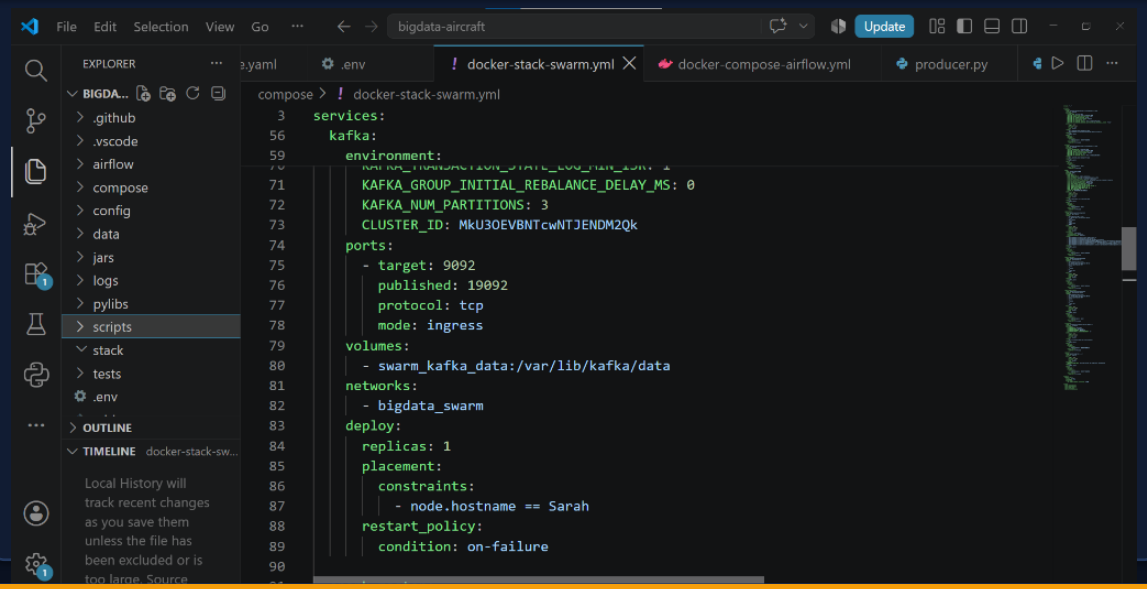
BATCH SUMMARY OUTPUT (UNITS 1–6)

- **Engine 1 (100 cycles):** 433,325 rows | Avg RUL: 49.45 | Max: 99
- **Engine 2 (75 cycles):** 1,049,088 rows | Avg RUL: 37.00 | Max: 74
- **Engine 3 (100 cycles):** 893,054 rows | Avg RUL: 49.07 | Max: 99
- **Engine 4 (95 cycles):** 414,172 rows | Avg RUL: 47.45 | Max: 94
- **Engine 5 (89 cycles):** 1,280,147 rows | Avg RUL: 44.11 | Max: 88
- **Engine 6 (94 cycles):** 836,850 rows | Avg RUL: 47.58 | Max: 93

Deployment, Testing & Honest Project Limitations

Docker Swarm & Tailscale setup, automated unit tests, and current technical limitations

DOCKER SWARM CONFIG (`compose/docker-stack-swarm.yml`)



MULTI-NODE DESIGN VS. ACTUAL RUNNING STATUS

- **Configured 3-Node Swarm Plan:** Designed to split containers across 3 team laptops connected by a private Tailscale VPN network (`mtu: 1200`).
- **Actual Running Status Today (Honest Report):** Currently running on 1 local Docker Desktop node (`Tailscale is stopped`), where all 11 containers (Airflow, Spark, HDFS, Kafka, ClickHouse, Superset) are up and healthy via Docker Compose.

AUTOMATED UNIT TESTS & CI (`tests/test_pipeline.py`)

- **GitHub Actions CI (`.github/workflows/ci.yml`):** Automatically runs `ruff` code checks and `pytest` on every push to `main`.
- **4 Verified Unit Tests:** 1. HDF5-to-Parquet chunk conversion and column names check

2. Kafka JSON message format and corrupted message injection
3. Disjoint engine-unit split check (zero data leakage)
4. Dead-Letter Queue (DLQ) filter check for invalid messages

CURRENT LIMITATIONS & REMAINING ISSUES

- **1. Single-Machine Memory Limits:** To avoid RAM crashes on 1 laptop, ML training samples 10% of rows (`--sample-ratio 0.1`) and ClickHouse loads 50,000 predictions per run.
- **2. Airflow Step 10 Network Name:** In local Compose mode, Step 10 fails if it looks for network `aircraft_bigdata_swarm` instead of `compose_default`.
- **3. DS03-012 Not Yet in Default DAG:** DS03-012 (9.82M rows) is converted to Parquet on disk, but the Airflow DAG currently runs on DS01-005 by default.
- **4. Replayed Data, Not Live Flight:** Kafka streams historical NASA data rather than live sensors from a real airplane.

Conclusion, Verified Achievements & Future Work

Summary of what we built and tested, plus realistic next steps for the project

WHAT WE SUCCESSFULLY ACHIEVED

- **Complete End-to-End Big Data Pipeline:** Connected NASA HDF5 data (6.12 GB, 17.46M rows) -> Parquet -> HDFS -> Spark Batch & Streaming -> Spark MLlib -> Airflow -> ClickHouse -> Superset.
- **Accurate RUL Prediction Without Leakage:** Split engines by unit ID and trained 3 models. Random Forest won with RMSE = 6.36 (R^2 = 0.945) on unseen Engine 6 and RMSE = 7.51 (R^2 = 0.906) across 2.73M test rows.
- **Reliable Streaming with Quarantine (DLQ):** Verified 41,010 Kafka messages, 30-second window stats, HDFS quarantine for bad JSON, and automatic critical alerts.
- **Interactive Dashboard in Superset:** Loaded 600,000 predictions and live alerts into ClickHouse and built visual charts in Apache Superset.

REALISTIC FUTURE WORK

- **1. Train on Both DS01 & DS03 Together:** Update the Airflow DAG to train on 100% of both DS01-005 and DS03-012 across multiple Swarm machines.
- **2. Live ML Prediction Inside Spark Streaming:** Load the saved Random Forest model inside `streaming_job.py` to predict RUL directly on incoming Kafka messages.
- **3. Fix Step 10 Network Auto-Detection:** Update Step 10 in Airflow so it automatically detects `compose_default` vs. `aircraft_bigdata_swarm`.
- **4. Try Deep Learning (LSTM / GRU):** Test sequence deep learning models to capture long-term sensor trends across flight cycles.

DATASET & TECHNOLOGY REFERENCES

1. Dataset: Chao, M. A., Kulkarni, C., Goebel, K., & Fink, O. (2021). "Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics." Data, 6(1), 5. NASA Ames Prognostics Data Repository (N-CMAPSS DS01-005 & DS03-012).
2. Big Data Stack: Apache Spark 4.1.3 (SQL, Structured Streaming, MLlib), Apache Kafka 4.0.0 (KRaft), Hadoop HDFS 3.2.1, Apache Airflow 2.9.1, Apache Parquet, ClickHouse 26.9, & Apache Superset 3.1.0.